

Exercício prático de Computação Gráfica

Modelagem de objetos simples, transformações lineares e manipulação da câmera

Atividade integrada aos fundamentos da Aula 3: representação geométrica, sistemas de coordenadas, pipeline gráfico, projeção e visualização. A implementação usa Python e Pygame para tornar observáveis as etapas que levam de um modelo geométrico a uma imagem na tela.

1. Visão geral

Item	Descrição
Público	Estudantes da disciplina de Computação Gráfica
Duração sugerida	2 a 3 horas
Formato	Individual ou em duplas
Produto	Programa executável, captura de tela e respostas reflexivas
Tecnologias	Python 3 e Pygame

2. Objetivos de aprendizagem

- Representar objetos simples por vértices, arestas e faces em coordenadas 3D.
- Aplicar translação, rotação e escala usando matrizes e vetores homogêneos.
- Separar coordenadas do modelo, do mundo, da câmera e da tela.
- Manipular a câmera virtual por teclado, alterando posição, orientação e distância de visualização.
- Comparar projeções perspectiva e ortográfica e relacioná-las ao pipeline gráfico.
- Organizar o programa em funções independentes, facilitando testes e extensão.

3. Enunciado

Desenvolva uma aplicação interativa que apresente uma pequena cena 3D formada por objetos simples — um cubo e uma pirâmide — projetados em uma janela 2D do Pygame. Cada objeto deverá possuir seu próprio modelo geométrico, transformação e cor. O usuário deverá poder transformar os objetos e movimentar a câmera para observar como as coordenadas e a projeção alteram a imagem final.

O Pygame será utilizado para a janela, os eventos e a rasterização das linhas/polígonos. A matemática de transformação e projeção será implementada no próprio programa, evidenciando a lógica do pipeline gráfico:

modelo → transformação do objeto → transformação de câmera → projeção → conversão para tela → desenho

4. Conceitos mobilizados

Conceito	Aplicação no exercício
Modelagem	Listas de vértices e faces para cubo e pirâmide.
Transformação linear	Matrizes de escala, rotação e translação.
Coordenadas	Conversão entre modelo, mundo, câmera e tela.
Câmera virtual	Posição, alvo e orientação simplificadas por deslocamento e rotação.

Projeção	Alternância entre perspectiva e ortográfica.
Rasterização	Desenho de arestas e faces com funções do Pygame.

5. Preparação do ambiente

No terminal, crie uma pasta para a atividade e instale a biblioteca. Recomenda-se utilizar um ambiente virtual para manter as dependências isoladas.

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# Linux/macOS
source .venv/bin/activate
python -m pip install pygame
python -c "import pygame; print(pygame.version.ver)"
```

Salve o código da seção seguinte como `atividade_camera.py` e execute com:

```
python atividade_camera.py
```

6. Código-base completo

O código abaixo é um ponto de partida funcional. Ele usa matrizes 4×4 para as transformações e realiza uma projeção simples em perspectiva ou ortográfica. Complete as extensões solicitadas depois de compreender cada função.

```
import math
import pygame
from pygame.locals import K_a, K_d, K_w, K_s, K_q, K_e, K_EQUALS, K_MINUS

WIDTH, HEIGHT = 960, 640
FPS = 60

# ----- Matrizes e vetores -----
def mat_mul(A, B):
    """Multiplica duas matrizes 4x4."""
    return [[sum(A[i][k] * B[k][j] for k in range(4))
              for j in range(4)] for i in range(4)]

def transform_point(M, p):
    """Aplica uma matriz 4x4 ao ponto 3D p=(x,y,z)."""
    x, y, z = p
    v = [x, y, z, 1.0]
    r = [sum(M[i][j] * v[j] for j in range(4)) for i in range(4)]
    w = r[3] if abs(r[3]) > 1e-8 else 1.0
    return (r[0] / w, r[1] / w, r[2] / w)

def identity():
    return [[1,0,0,0], [0,1,0,0], [0,0,1,0], [0,0,0,1]]

def translation(tx, ty, tz):
    M = identity(); M[0][3], M[1][3], M[2][3] = tx, ty, tz
    return M

def scale(sx, sy, sz):
```

```

    return [[sx,0,0,0], [0,sy,0,0], [0,0,sz,0], [0,0,0,1]]

def rotation_x(a):
    c, s = math.cos(a), math.sin(a)
    return [[1,0,0,0], [0,c,-s,0], [0,s,c,0], [0,0,0,1]]

def rotation_y(a):
    c, s = math.cos(a), math.sin(a)
    return [[c,0,s,0], [0,1,0,0], [-s,0,c,0], [0,0,0,1]]

def rotation_z(a):
    c, s = math.cos(a), math.sin(a)
    return [[c,-s,0,0], [s,c,0,0], [0,0,1,0], [0,0,0,1]]

# ----- Modelos geométricos -----
def cube(size=2.0):
    h = size / 2
    vertices = [(-h,-h,-h), (h,-h,-h), (h,h,-h), (-h,h,-h),
                (-h,-h,h), (h,-h,h), (h,h,h), (-h,h,h)]
    faces = [(0,1,2,3), (4,7,6,5), (0,4,5,1),
             (3,2,6,7), (1,5,6,2), (0,3,7,4)]
    return vertices, faces

def pyramid(size=2.0, height=2.2):
    h = size / 2
    vertices = [(-h,0,-h), (h,0,-h), (h,0,h), (-h,0,h), (0,height,0)]
    faces = [(0,1,2,3), (0,4,1), (1,4,2), (2,4,3), (3,4,0)]
    return vertices, faces

# ----- Câmera e projeção -----
def project(point, camera_z, perspective=True):
    x, y, z = point
    z_view = z - camera_z
    # A câmera olha para +Z; pontos atrás/próximos demais são descartados.
    if z_view <= 0.1:
        return None
    if perspective:
        focal = 520.0
        sx = focal * x / z_view
        sy = focal * y / z_view
    else:
        sx = 230.0 * x
        sy = 230.0 * y
    return (int(WIDTH/2 + sx), int(HEIGHT/2 - sy), z_view)

def draw_model(screen, vertices, faces, model_matrix, camera_z, perspective, color):
    world = [transform_point(model_matrix, v) for v in vertices]
    projected = [project(v, camera_z, perspective) for v in world]
    for face in faces:
        points = [projected[i] for i in face]
        if all(p is not None for p in points):
            # Tom mais escuro para diferenciar as faces sem iluminação real.
            fill = tuple(max(25, c - 55) for c in color)
            pygame.draw.polygon(screen, fill, [(p[0], p[1]) for p in points])
            pygame.draw.lines(screen, color, True,
                              [(p[0], p[1]) for p in points], 2)

```

```

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption('Computação Gráfica – Modelagem e Câmera')
    clock = pygame.time.Clock()
    font = pygame.font.SysFont('Arial', 18)

    cube_v, cube_f = cube()
    pyr_v, pyr_f = pyramid()
    angle_x = angle_y = angle_z = 0.0
    camera_z = -8.0
    perspective = True
    running = True

    while running:
        dt = clock.tick(FPS) / 1000.0
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    running = False
                elif event.key == pygame.K_p:
                    perspective = not perspective

        keys = pygame.key.get_pressed()
        speed = 2.0 * dt
        if keys[K_a]: angle_y -= speed
        if keys[K_d]: angle_y += speed
        if keys[K_w]: angle_x -= speed
        if keys[K_s]: angle_x += speed
        if keys[K_q]: angle_z -= speed
        if keys[K_e]: angle_z += speed
        if keys[K_EQUALS] or keys[pygame.K_KP_PLUS]: camera_z += 3.0 * dt
        if keys[K_MINUS] or keys[pygame.K_KP_MINUS]: camera_z -= 3.0 * dt

        screen.fill((12, 20, 35))
        # Ordem: escala -> rotações -> translação (composição matricial).
        R = mat_mul(rotation_z(angle_z), mat_mul(rotation_y(angle_y), rotation_x(angle_x)))
        cube_model = mat_mul(translation(-2.0, 0.0, 7.0), mat_mul(R, scale(1.2, 1.2, 1.2)))
        pyramid_model = mat_mul(translation(2.0, -1.0, 9.0), mat_mul(R, scale(1.0, 1.0, 1.0)))
        draw_model(screen, cube_v, cube_f, cube_model, camera_z, perspective, (80, 190, 255))
        draw_model(screen, pyr_v, pyr_f, pyramid_model, camera_z, perspective, (255, 170, 80))

        mode = 'perspectiva' if perspective else 'ortográfica'
        info = font.render(f'Projeção: {mode} | Câmera Z: {camera_z:.1f} | P: alternar', True,
(235,235,235))
        help_text = font.render('W/S X A/D Y Q/E Z +/- câmera ESC sair', True, (180,200,220))
        screen.blit(info, (18, 16)); screen.blit(help_text, (18, 42))
        pygame.display.flip()

    pygame.quit()

if __name__ == '__main__':
    main()

```

7. Controles e observações

Tecla	Ação
W / S	Rotacionar em torno do eixo X
A / D	Rotacionar em torno do eixo Y
Q / E	Rotacionar em torno do eixo Z
+ / -	Aproximar ou afastar a câmera
P	Alternar perspectiva e ortográfica
ESC	Encerrar a aplicação

A convenção adotada coloca a câmera em uma posição negativa no eixo Z e os objetos em valores positivos. A função `project()` descarta pontos com profundidade insuficiente, evitando divisões instáveis e representando o recorte do volume de visão.

8. Roteiro de desenvolvimento e investigação

1. Execute o código sem alterações e identifique os dois objetos na cena.
2. Comente temporariamente a translação da pirâmide e observe a sobreposição dos modelos.
3. Altere a ordem das rotações e descreva a diferença visual.
4. Troque a escala apenas do cubo. O que muda: posição, orientação ou tamanho?
5. Aproxime e afaste a câmera. Compare o tamanho aparente na perspectiva.
6. Alterne para a projeção ortográfica e explique por que a profundidade deixa de alterar a escala.
7. Adicione um terceiro objeto simples, como um prisma ou uma segunda pirâmide.
8. Inclua uma legenda dos eixos ou uma grade no plano XZ.

9. Questões para discussão

9. Qual é a diferença entre as coordenadas de um vértice no modelo e suas coordenadas na tela?
10. Por que a composição das transformações depende da ordem em que as matrizes são multiplicadas?
11. Como a divisão pela profundidade produz o efeito de perspectiva?
12. O que ocorre quando a câmera se aproxima demais de um objeto?
13. Em que sentido a manipulação de `camera_z` representa uma transformação de câmera?
14. Qual projeção é mais apropriada para medir dimensões sem deformação aparente? Justifique.
15. Quais partes de um pipeline gráfico completo não foram implementadas neste exercício?

10. Extensões opcionais

- Implementar movimentação lateral e vertical da câmera, com X e Y.
- Adicionar rotação da câmera para simular olhar para os lados e para cima/baixo.
- Implementar ordenação por profundidade (painter's algorithm) para desenhar faces corretamente.
- Calcular uma cor diferente para cada face usando uma normal aproximada.
- Adicionar um modo de visualização apenas em arame (wireframe).
- Criar uma matriz de visão baseada em posição da câmera e ponto-alvo.
- Permitir salvar uma captura da cena ao pressionar uma tecla.

11. Entrega

Entregue o código-fonte, uma captura de tela da aplicação em execução e um texto curto (aproximadamente uma página) explicando o pipeline implementado e as diferenças observadas entre as projeções.

12. Síntese conceitual

A atividade consolida a passagem do modelo geométrico para a imagem exibida. Os objetos começam como vértices definidos em seu próprio sistema de coordenadas; as matrizes posicionam, dimensionam e orientam esses vértices no mundo; a câmera estabelece o ponto de vista; a projeção reduz a informação tridimensional a coordenadas de tela; e o Pygame rasteriza as primitivas resultantes. Essa sequência prepara a compreensão de cenas 3D e de aplicações interativas de realidade virtual e aumentada.